



UNIDAD 1:

SELECCIÓN DE ARQUITECTURAS Y HERRAMIENTAS WEB

Desarrollo Web en Entorno Cliente (DWEC)

Contenido

1. Evolución y características de los navegadores web	3
1.1 El modelo Cliente/Servidor en la Web	3
1.2 Navegadores actuales y cuota de mercado	3
1.3 Herramientas de desarrollo integradas (DevTools)	4
2. Arquitectura de ejecución en el cliente	4
2.1 Componentes de la arquitectura de referencia del navegador	5
2.2 Motores de renderizado y motores de JavaScript actuales	5
2.3 Conceptos de arquitectura frontend moderna	6
3. Lenguajes y tecnologías en el entorno cliente	6
3.1 HTML5 y CSS3	7
3.2 JavaScript moderno (ES6+) y TypeScript	7
3.3 WebAssembly (WA)	8
4. Comunicación asíncrona y almacenamiento en el cliente	8
4.1 Comunicación asíncrona: AJAX, XMLHttpRequest y API Fetch	8
4.2 Intercambio de datos: JSON vs XML	9
4.3 Almacenamiento local en el cliente	9
5. Ecosistema y herramientas de desarrollo (Tooling)	9
5.1 Entornos de ejecución y gestores de paquetes	9
5.2 Herramientas de construcción y empaquetado (Bundlers)	10
5.3 Herramientas de calidad de código	10
5.4 Introducción a los frameworks de cliente	10

En este primer tema presentamos los conceptos fundamentales para comprender el contexto de ejecución de las aplicaciones web en el entorno cliente. Analizaremos el modelo cliente/servidor, la evolución y funcionamiento de los navegadores web modernos, la arquitectura interna de ejecución, los lenguajes estándar (HTML5, CSS3, JavaScript ES6+, TypeScript y WebAssembly), la comunicación asíncrona, las tecnologías de almacenamiento local y el ecosistema de herramientas de desarrollo frontend.

1. Evolución y características de los navegadores web

1.1 El modelo Cliente/Servidor en la Web

La World Wide Web (WWW) se fundamenta en una arquitectura distribuida basada en el modelo **Cliente/Servidor**:

- **Cliente Web:** Proceso o dispositivo consumidor de servicios (principalmente **el navegador web**). Envía peticiones (*requests*) de recursos a través de protocolos de red.
- **Servidor Web:** Proceso proveedor de servicios que escucha peticiones, procesa la lógica del servidor (o consulta bases de datos) y responde (*responses*) enviando documentos, código o datos al cliente.

La comunicación entre cliente y servidor se realiza principalmente mediante el protocolo HTTP/HTTPS, donde los mensajes intercambiados constituyen el elemento de acoplamiento entre ambos extremos. En las aplicaciones web modernas, el cliente ha pasado de ser un simple visor estático a un entorno de ejecución complejo capaz de procesar lógica de negocio, gestionar el estado local y renderizar interfaces dinámicas.

1.2 Navegadores actuales y cuota de mercado

Un navegador web (browser) es la aplicación cliente encargada de solicitar, interpretar y renderizar recursos web identificados mediante direcciones URL (*Uniform Resource Locator*). Tras la discontinuación de navegadores heredados como Internet Explorer, el panorama actual se concentra en cuatro grandes navegadores multiplataforma:

- **Google Chrome:** Basado en el proyecto de código abierto Chromium. Es el navegador con mayor cuota de mercado a nivel mundial debido a su rendimiento, estabilidad mediante arquitectura multiproceso y amplio ecosistema de extensiones.
- **Mozilla Firefox:** Navegador de código abierto desarrollado por la Fundación Mozilla. Destaca por su compromiso con la privacidad del usuario, soporte estricto de estándares web y herramientas de desarrollo avanzadas.
- **Microsoft Edge:** Basado en Chromium. Ofrece integración nativa con el sistema operativo Windows, optimización de recursos y herramientas de productividad.
- **Apple Safari:** Navegador predeterminado en los sistemas macOS e iOS. Altamente optimizado para la eficiencia energética y el rendimiento en hardware de Apple.

En [gs.StatCounter.com](https://gs.statcounter.com) podemos ver la tasa de uso mundial de los distintos navegadores web.

Criterios de clasificación de los navegadores actuales:

- **Plataforma y soporte de SO:** Compatibilidad con sistemas de escritorio (Windows, macOS, Linux) y móviles (Android, iOS).
- **Soporte de Estándares Web:** Grado de adopción de las últimas especificaciones de W3C y WHATWG para HTML5, CSS3 y ECMAScript.
- **Seguridad y Privacidad:** Bloqueo de rastreadores de terceros, protección contra phishing/malware y aislamiento de procesos (*sandboxing*).
- **Licencia y Modelo de Desarrollo:** Software de código abierto (Chromium, Firefox) frente a distribuciones propietarias comercializadas por corporaciones.

1.3 Herramientas de desarrollo integradas (DevTools)

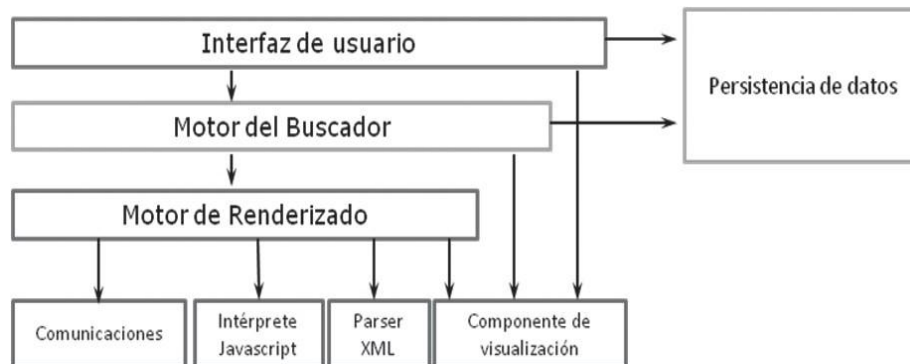
Los navegadores modernos incorporan un conjunto indispensable de herramientas para la inspección, depuración y optimización de aplicaciones web, conocidas como **DevTools** (Developer Tools). Se accede a ellas mediante la tecla F12 o el atajo Ctrl + Shift + I (Cmd + Option + I en macOS).

- **Elementos (Elements / Inspector):** Permite inspeccionar y editar en tiempo real la estructura del DOM (HTML) y los estilos aplicados (CSS).
- **Consola (Console):** Entorno interactivo para ejecutar código JavaScript, visualizar mensajes de depuración (`console.log`, `console.error`) y registrar excepciones no capturadas.
- **Fuentes / Depurador (Sources / Debugger):** Permite inspeccionar los archivos de código fuente, establecer puntos de interrupción (breakpoints), ejecutar código paso a paso y supervisar variables.
- **Red (Network):** Monitoriza todas las peticiones HTTP/HTTPS realizadas por la página (imágenes, scripts, llamadas a API REST/Fetch), mostrando tiempos de carga, cabeceras, estados de respuesta y tamaño de recursos.
- **Aplicación / Memoria (Application / Storage):** Permite gestionar el almacenamiento en el cliente: Cookies, LocalStorage, SessionStorage, IndexedDB y Service Workers.
- **Rendimiento y Auditorías (Performance / Lighthouse):** Analiza la velocidad de renderizado, el uso de CPU/memoria y genera informes de accesibilidad, SEO y buenas prácticas.

2. Arquitectura de ejecución en el cliente

2.1 Componentes de la arquitectura de referencia del navegador

La arquitectura interna de un navegador web moderno se compone de varios subsistemas interconectados:



- **1. Interfaz de usuario:** Incluye los elementos visuales del navegador ajenos a la ventana del sitio web (barra de direcciones, botones de navegación, marcadores, pestañas).
- **2. Motor de navegación (Browser Engine):** Administra las acciones entre la interfaz de usuario y el motor de renderizado.
- **3. Motor de renderizado (Rendering Engine):** Subsistema principal encargado de interpretar documentos **HTML y CSS** para dibujar la representación visual en pantalla.
- **4. Subsistema de comunicaciones:** Gestiona las peticiones de red (**HTTP/HTTPS, WebSockets**) e identifica la codificación de los datos recibidos mediante tipos MIME.
- **5. Intérprete / Motor de JavaScript:** Subsistema especializado en parsear, compilar (JIT) y ejecutar el código **JavaScript/TypeScript**.
- **6. Parser XML/HTML:** Transforma el marcado de texto en una estructura jerárquica de árbol en memoria conocida como **DOM** (Document Object Model).
- **7. Componente de visualización (UI Backend):** Proporciona primitivas de dibujo básicas de la plataforma o sistema operativo subyacente.
- **8. Persistencia de datos:** Almacena en el disco local los datos de navegación, **cookies, caché de recursos y bases de datos** del cliente (**Web Storage, IndexedDB**).

2.2 Motores de renderizado y motores de JavaScript actuales

En la actualidad, el desarrollo web está configurado por tres grandes motores de renderizado y sus correspondientes motores de ejecución JavaScript:

Navegador / Plataforma	Motor de Renderizado	Motor de JavaScript
Google Chrome / MS Edge / Opera	Blink (Fork de WebKit)	V8
Mozilla Firefox	Gecko	SpiderMonkey
Apple Safari	WebKit	JavaScriptCore (Nitro)

- **Blink:** Motor de renderizado altamente optimizado utilizado por el proyecto Chromium.
- **V8:** Motor de JavaScript desarrollado en C++ por Google, utilizado tanto en Chrome como fuera del navegador en entornos de ejecución como Node.js y Deno. Utiliza compilación Just-In-Time (JIT) para transformar código JS directamente a código máquina.

2.3 Conceptos de arquitectura frontend moderna

- **MPA (Multi-Page Application):** Arquitectura tradicional donde cada acción del usuario requiere solicitar una nueva página completa al servidor, el cual devuelve un documento HTML renderizado.
- **SPA (Single-Page Application):** Aplicación de una sola página donde el servidor entrega una plantilla HTML básica y el navegador carga los scripts necesarios. Toda la navegación posterior se realiza dinámicamente sin recargar la página completa, solicitando únicamente datos (JSON) mediante llamadas asíncronas.
- **CSR (Client-Side Rendering):** El renderizado de la interfaz ocurre íntegramente en el navegador del cliente mediante JavaScript.
- **SSR (Server-Side Rendering):** El servidor genera el HTML con el contenido ya listo para cada petición, lo que mejora el tiempo de primera carga y la indexación en motores de búsqueda (SEO).
- **PWA (Progressive Web App):** Aplicación web que utiliza APIs modernas (Service Workers) para ofrecer características similares a las aplicaciones nativas: funcionamiento offline, notificaciones push e instalación en el sistema operativo.

3. Lenguajes y tecnologías en el entorno cliente

3.1 HTML5 y CSS3

- **HTML5 (HyperText Markup Language):** Es el lenguaje de marcado estándar para estructurar el contenido de las páginas web. HTML5 introdujo etiquetas semánticas (<header>, <nav>, <article>, <section>, <footer>), elementos multimedia nativos (<audio>, <video>), soporte para gráficos vectoriales (<svg>, <canvas>) y APIs de integración.
- **CSS3 (Cascading Style Sheets):** Lenguaje de hojas de estilo utilizado para describir la presentación visual de un documento HTML. CSS3 aporta capacidades avanzadas como diseño flexible (Flexbox), cuadrículas bidimensionales (CSS Grid), variables CSS (propiedades personalizadas), animaciones, transiciones y consultas de medios (Media Queries) para garantizar un diseño responsivo adaptado a múltiples dispositivos.

3.2 JavaScript moderno (ES6+) y TypeScript

JavaScript (ECMAScript): Es el lenguaje de programación estándar interpretable por todos los navegadores web. A partir de la especificación ECMAScript 2015 (ES6), el lenguaje incorporó mejoras fundamentales:

- Es un lenguaje de secuencias de comandos basado en objetos, estructurado e interpretado.
- Débilmente tipado.
- Permite creación de clases y herencia.
- Se usa principalmente en el lado del cliente, aunque también se puede usar en el lado del servidor: Node.js, Deno.js,
- Tipado dinámico: el tipo está asociado al valor.
- Formado casi en su totalidad por objetos, que son realmente arrays asociativos.
- Permite evaluación de expresiones regulares en tiempo de ejecución (uso en validación de formularios, por ejemplo).
- Existen gran cantidad de frameworks con gran aceptación y expansión en el mundo laboral. Por ejemplo: jQuery, Angular, React.js, Ember.js, Meteor.js, Vue.js, Mercury.js, Foundation.js, Backbone.js, Modernizr, Video.js, Maps.js, etc.
- Permite el control de eventos, incluidos eventos táctiles.
- Realiza conexiones asíncronas con el servidor.

TypeScript: Es un superconjunto de JavaScript desarrollado por Microsoft que añade tipado estático opcional y características avanzadas de programación orientada a objetos. TypeScript se transpila a JavaScript estándar para que pueda ser ejecutado por cualquier navegador. Aporta gran solidez en el desarrollo de aplicaciones frontend de gran escala, detectando errores en tiempo de compilación e integrándose con entornos de desarrollo modernos.

3.3 WebAssembly (WA)

WebAssembly es un formato de instrucción binaria portátil de bajo nivel diseñado como estándar web abierto. Permite ejecutar código escrito en lenguajes de alto nivel como C++, Rust, Go o C# dentro del navegador a una velocidad cercana a la nativa.

WebAssembly no sustituye a JavaScript, sino que complementa su ecosistema, permitiendo ejecutar en el entorno cliente tareas intensivas de cómputo como procesamiento de imagen/vídeo, videojuegos 3D, emuladores y criptografía.

4. Comunicación asíncrona y almacenamiento en el cliente

4.1 Comunicación asíncrona: AJAX, XMLHttpRequest y API Fetch

La comunicación asíncrona permite a la aplicación web intercambiar datos con el servidor en segundo plano sin necesidad de recargar la página completa.

- **AJAX (Asynchronous JavaScript and XML):** Término tradicional que engloba el conjunto de técnicas para actualizar partes de una página web dinámicamente.
- **XMLHttpRequest:** Objeto histórico de JavaScript que permitió el nacimiento de las aplicaciones AJAX. Aunque sigue estando soportado por compatibilidad, su sintaxis basada en eventos resulta verbosa y compleja.
- **API Fetch:** Interfaz moderna, basada en Promesas (*promises*), que reemplaza a XMLHttpRequest. Proporciona un método limpio y legible para realizar peticiones HTTP/HTTPS asíncronas.

Ejemplo de petición Fetch con async/await:

```
async function obtenerDatos() {
  try {
    const response = await fetch('https://api.ejemplo.com/productos');
    if (!response.ok)
      throw new Error('Error en la respuesta de la red');
    const datos = await response.json();
    console.log(datos);
  }
  catch (error) {
    console.error('Error al realizar la petición:', error);
  }
}
```

4.2 Intercambio de datos: JSON vs XML

- **JSON (JavaScript Object Notation):** Es el formato estándar de intercambio de datos en la web actual. Es liviano, fácil de leer por los humanos y se analiza de forma nativa en JavaScript sin sobrecarga de cómputo (*JSON.parse()* y *JSON.stringify()*).
- **XML (Extensible Markup Language):** Formato estructurado basado en etiquetas. Aunque fue muy utilizado en el origen de AJAX, en el desarrollo frontend moderno ha sido sustituido en su mayor parte por JSON debido a la mayor complejidad de procesamiento de XML.

4.3 Almacenamiento local en el cliente

Las aplicaciones web modernas disponen de mecanismos para guardar datos directamente en el navegador del usuario:

- **LocalStorage (Web Storage API):** Almacena datos clave-valor de forma persistente. Los datos no expiran al cerrar el navegador y permanecen hasta que la aplicación o el usuario los elimine explícitamente (límite aprox. 5-10 MB por origen).
- **SessionStorage (Web Storage API):** Almacena datos clave-valor para la sesión actual. Los datos se borran automáticamente al cerrar la pestaña o ventana del navegador.
- **IndexedDB:** Base de datos transaccional orientada a objetos (NoSQL) integrada en el navegador. Permite almacenar grandes volúmenes de datos estructurados e índices, siendo ideal para aplicaciones con soporte offline (PWA).
- **Cookies:** Pequeños fragmentos de texto enviados entre el cliente y el servidor en cada petición HTTP. Se utilizan principalmente para la gestión de sesiones de usuario y autenticación (HttpOnly, Secure, SameSite), no para el almacenamiento masivo de datos debido a su límite de capacidad (~4 KB) y su impacto en el ancho de banda.

5. Ecosistema y herramientas de desarrollo (Tooling)

5.1 Entornos de ejecución y gestores de paquetes

El desarrollo web frontend actual requiere herramientas de línea de comandos para la gestión de dependencias y automatización de tareas:

- **Node.js:** Entorno de ejecución de JavaScript fuera del navegador (basado en el motor V8). Es la base sobre la que se ejecutan los compiladores, empaquetadores y servidores de desarrollo frontend.
- **npm (Node Package Manager):** El gestor oficial de Node.js y el registro de paquetes JavaScript más amplio del mundo.
- **pnpm / Yarn:** Alternativas modernas enfocadas en la optimización del espacio en disco, velocidad de instalación y gestión eficiente de monorepositorios.

5.2 Herramientas de construcción y empaquetado (Bundlers)

Las aplicaciones web modernas no se distribuyen en producción como archivos sueltos, sino que requieren procesos de transpilación (TypeScript a JS), minificación, optimización de recursos y empaquetado (*bundling*):

- **Vite:** Herramienta de construcción de nueva generación basada en módulos ES nativos (ESM) para el desarrollo ultrarrápido y empaquetado eficiente de producción.
- **Webpack:** Empaquetador tradicional altamente configurable y con un amplio ecosistema de loaders y plugins.

5.3 Herramientas de calidad de código

- **ESLint:** Herramienta de análisis estático (linter) que identifica patrones problemáticos, errores sintácticos y vulnerabilidades en el código JavaScript/TypeScript.
- **Prettier:** Formateador de código automático que garantiza un estilo de código limpio y uniforme en todo el equipo de desarrollo.

5.4 Introducción a los frameworks de cliente

En el desarrollo web, los frameworks y las APIs son herramientas esenciales que facilitan la creación de aplicaciones robustas, escalables y eficientes. Ambas tecnologías permiten a los desarrolladores escribir menos código repetitivo y enfocarse en la lógica de negocio de sus aplicaciones.

¿Qué es un Framework?

Un framework es un conjunto de herramientas, bibliotecas y estructuras predefinidas que facilitan el desarrollo de aplicaciones web. Proporciona una arquitectura base, siguiendo patrones de diseño que ayudan a organizar el código de manera eficiente.

¿Cómo funciona un Framework?

Un framework actúa como una plantilla estructurada en la que los desarrolladores solo necesitan agregar su lógica específica. Normalmente, sigue el patrón **MVC (Modelo-Vista-Controlador)**, que separa la lógica de datos, la presentación y la interacción del usuario.

En el desarrollo de aplicaciones web complejas, es habitual utilizar frameworks o librerías frontend para estructurar la interfaz mediante componentes reutilizables y reactivos:

-
- **React.js:** Librería declarativa desarrollada por Meta basada en componentes y el uso del DOM Virtual.
 - **Vue.js:** Framework progresivo, ligero y de fácil aprendizaje con reactividad transparente.
 - **Angular:** Framework completo y estructurado mantenido por Google, escrito nativamente en TypeScript e ideal para aplicaciones empresariales.

¿Qué es una API?

Una API (Application Programming Interface) es un conjunto de reglas y definiciones que permiten que diferentes aplicaciones se comuniquen entre sí. En programación web, las APIs permiten que los clientes (navegadores, aplicaciones móviles, etc.) soliciten datos al servidor y reciban respuestas en formatos como **JSON** o **XML**.

¿Cómo funciona una API en el desarrollo web?

1. Un usuario hace una solicitud desde el navegador o una aplicación móvil.
2. El servidor recibe la solicitud y ejecuta una operación (por ejemplo, consultar una base de datos).
3. El servidor responde con datos en formato **JSON** o **XML**.
4. La aplicación cliente interpreta los datos y los muestra en la interfaz de usuario.

Tipos de APIs más comunes en la web

- **RESTful APIs:** Basadas en HTTP y usan métodos como GET, POST, PUT y DELETE.
- **GraphQL:** Permite consultas más flexibles y eficientes, por ejemplo, se puede definir qué información se desea recibir y en qué formato. Está desarrollado por Facebook y liberado como código abierto en 2015, aunque desde 2018 está gestionado por la Linux Foundation. Facebook lo usa en multitud de productos, como sus aplicaciones web y por supuesto las apps de Android e iOS. Pero además, otras muchas empresas en el mundo ofrecen servicios web basados en GraphQL, como Pinterest o Github.
- **WebSockets:** Para comunicación en tiempo real (chats, videollamadas, notificaciones, etc.).

Debe quedar clara la diferencia entre framework y API, mientras que los frameworks simplifican el desarrollo de aplicaciones proporcionando estructuras organizadas, mientras que las APIs permiten la comunicación eficiente entre sistemas. Ambos son fundamentales para construir aplicaciones web modernas, escalables y seguras.